JAVA COLLECTIONS FRAMEWORK

Student Programming Activities | Module 6

Name: Jian Dotusme Section: TN22-3 Date: 10/20/2026

General Instructions

- Complete all three activities as separate, console-based Java programs. Write your own solution; no starter code is provided.
- Every program must use Scanner for user input, a repeating numbered menu, methods, loops, conditional statements, and the required collection types.
- Validate invalid menu entries and empty inputs. Keep the program running until Exit is selected. Use Java generics rather than raw collection types.
- Submit three .java files, screenshots showing successful test runs, and written answers to the open-ended questions.

ACTIVITY 01 | Campus Lost-and-Found Tracker

Problem Statement

A campus office receives lost items throughout the day. Staff need to record items in the order they arrive and look up a particular report. Design a console program that manages these records without using a fixed-size array.

Required Java Collections

ArrayList<String> (required); use add(), get(), remove(), size(), and iteration.

Required Menu

1. Add an item description. 2. View all reports with numbered positions. 3. Search reports by a keyword (case-insensitive). 4. Remove a report by its displayed number. 5. Show total reports. 6. Exit.

Functional Requirements

- Accept multiword descriptions, e.g., “black umbrella near library”.
- Display a clear message when the list is empty or a search has no matches.
- Reject invalid removal positions without crashing.
- Preserve the order in which reports were entered.

Test Scenarios (demonstrate all) add screenshot for each testcases and corresponding output.

1. Add “Blue water bottle”, “Black umbrella”, and “Student ID card”; display the list.

2. Search for “umbrella” and “laptop”; show matching and no-match cases.

3. Remove report 2; verify remaining order and count.

4. Attempt to remove report 99 and handle it safely.

Open-Ended Questions

1. Why is ArrayList more appropriate than a fixed-length array for this scenario?

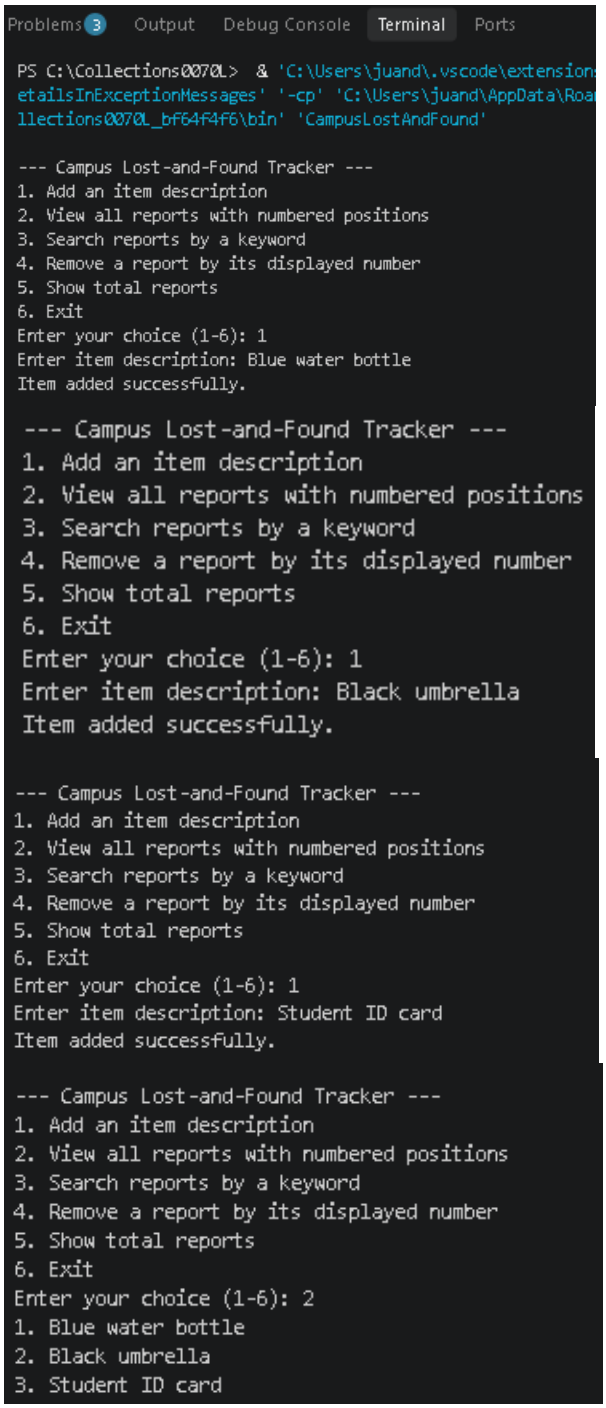
An ArrayList grows and shrinks automatically as you add or remove items, whereas a standard array has a fixed size that you can't change once it's created.

2. What happens to the indices of later items when an item is removed?

When an item gets removed, all the items after it shift one spot to the left, which decreases their index numbers by 1.

3. If each report needed an ID, date, and status, how would you improve the data model?

You could build a custom class (like a Report class) that stores the ID, date, status, and description together, and then make an ArrayList<Report> instead of just storing strings.


Problems 3
Output
Debug Console
Terminal
Ports
PS C:\Collections0070L> & 'C:\Users\juand\.vscode\extensions
etailsInExceptionMessages' '-cp' 'C:\Users\juand\AppData\Roa
llections0070L_bf64f4f6\bin' 'CampusLostAndFound'
--- Campus Lost-and-Found Tracker ---
1. Add an item description
2. View all reports with numbered positions
3. Search reports by a keyword
4. Remove a report by its displayed number
5. Show total reports
6. Exit
Enter your choice (1-6): 1
Enter item description: Blue water bottle
Item added successfully.
--- Campus Lost-and-Found Tracker ---
1. Add an item description
2. View all reports with numbered positions
3. Search reports by a keyword
4. Remove a report by its displayed number
5. Show total reports
6. Exit
Enter your choice (1-6): 1
Enter item description: Black umbrella
Item added successfully.
--- Campus Lost-and-Found Tracker ---
1. Add an item description
2. View all reports with numbered positions
3. Search reports by a keyword
4. Remove a report by its displayed number
5. Show total reports
6. Exit
Enter your choice (1-6): 1
Enter item description: Student ID card
Item added successfully.
--- Campus Lost-and-Found Tracker ---
1. Add an item description
2. View all reports with numbered positions
3. Search reports by a keyword
4. Remove a report by its displayed number
5. Show total reports
6. Exit
Enter your choice (1-6): 2
1. Blue water bottle
2. Black umbrella
3. Student ID card

```
--- Campus Lost-and-Found Tracker ---
1. Add an item description
2. View all reports with numbered positions
3. Search reports by a keyword
4. Remove a report by its displayed number
5. Show total reports
6. Exit
Enter your choice (1-6): 3
Enter keyword to search: umbrella
2. Black umbrella
```

```
--- Campus Lost-and-Found Tracker ---
1. Add an item description
2. View all reports with numbered positions
3. Search reports by a keyword
4. Remove a report by its displayed number
5. Show total reports
6. Exit
Enter your choice (1-6): 3
Enter keyword to search: laptop
No reports matched that keyword.
```

```
--- Campus Lost-and-Found Tracker ---
1. Add an item description
2. View all reports with numbered positions
3. Search reports by a keyword
4. Remove a report by its displayed number
5. Show total reports
6. Exit
Enter your choice (1-6): 2
1. Blue water bottle
2. Student ID card
```

```
--- Campus Lost-and-Found Tracker ---
1. Add an item description
2. View all reports with numbered positions
3. Search reports by a keyword
4. Remove a report by its displayed number
5. Show total reports
6. Exit
Enter your choice (1-6): 5
Total reports recorded: 2
```

```
--- Campus Lost-and-Found Tracker ---
1. Add an item description
2. View all reports with numbered positions
3. Search reports by a keyword
4. Remove a report by its displayed number
5. Show total reports
6. Exit
Enter your choice (1-6): 4
Enter report number to remove: 2
Removed: Black umbrella
```

```
--- Campus Lost-and-Found Tracker ---
1. Add an item description
2. View all reports with numbered positions
3. Search reports by a keyword
4. Remove a report by its displayed number
5. Show total reports
6. Exit
Enter your choice (1-6): 4
Enter report number to remove: 99
Invalid position number.
```

ACTIVITY 02 | Club Membership and Event Codes

Problem Statement

A student organization registers event codes. The same code must not be registered twice, even if the user types it using different capitalization. Design a system that prevents duplicate codes and allows membership checking.

Required Java Collections

HashSet<String> (required); use add(), contains(), remove(), size(), and enhanced for.

Required Menu

1. Register event code. 2. View all unique codes. 3. Check whether a code exists. 4. Remove a code. 5. Show total unique codes. 6. Exit.

Functional Requirements

- Normalize codes using trim() and toUpperCase().
- Reject blank codes and report duplicate attempts.
- Explain in your output that the display order is not guaranteed.
- Show a friendly message when the set is empty.

Test Scenarios (demonstrate all) add screenshot for each testcases and corresponding output.

1. Register “TECH101”, “tech101”, and “ART205”; verify only two unique codes remain.

2. Check “TECH101” and “MATH300”.

3. Remove “ART205”, then verify it no longer exists.

4. Attempt to remove an unknown code.

Open-Ended Questions

1. Why does HashSet fit the duplicate-prevention requirement?

A HashSet naturally rejects duplicates. If you try to add something that is already inside the set, it simply won't add it and returns false.

2. Why should students not rely on the printed order of a HashSet?

A HashSet organizes elements based on their internal hash values rather than the sequence you typed them in, so the print order looks random.

3. What collection would you choose if codes must stay in registration order, and why?

A LinkedHashSet, because it keeps track of the insertion order while still preventing duplicates.

```
PS C:\Collections0070L>  & 'C:\Users\juand\.vsc
' '-XX:+ShowCodeDetailsInExceptionMessages' '-c
7d9b2d6b0628\redhat.java\jdt_ws\Collections0070

--- Club Membership and Event Codes ---
1. Register event code
2. View all unique codes
3. Check whether a code exists
4. Remove a code
5. Show total unique codes
6. Exit
Enter your choice (1-6): 1
Enter event code: TECH101
Event code registered successfully.

--- Club Membership and Event Codes ---
1. Register event code
2. View all unique codes
3. Check whether a code exists
4. Remove a code
5. Show total unique codes
6. Exit
Enter your choice (1-6): 1
Enter event code: tech101
Duplicate code! It's already registered.

--- Club Membership and Event Codes ---
1. Register event code
2. View all unique codes
3. Check whether a code exists
4. Remove a code
5. Show total unique codes
6. Exit
Enter your choice (1-6): 1
Enter event code: ART205
Event code registered successfully.

--- Club Membership and Event Codes ---
1. Register event code
2. View all unique codes
3. Check whether a code exists
4. Remove a code
5. Show total unique codes
6. Exit
Enter your choice (1-6): 2
Registered Codes (Order is not guaranteed):
- TECH101
```

```
--- Club Membership and Event Codes ---
1. Register event code
2. View all unique codes
3. Check whether a code exists
4. Remove a code
5. Show total unique codes
6. Exit
Enter your choice (1-6): 3
Enter code to check: TECH101
Yes, that code exists in the system.

--- Club Membership and Event Codes ---
1. Register event code
2. View all unique codes
3. Check whether a code exists
4. Remove a code
5. Show total unique codes
6. Exit
Enter your choice (1-6): 3
Enter code to check: MATH300
No, that code does not exist.
```

```
--- Club Membership and Event Codes ---
1. Register event code
2. View all unique codes
3. Check whether a code exists
4. Remove a code
5. Show total unique codes
6. Exit
Enter your choice (1-6): 4
Enter code to remove: ART205
Code removed successfully.

--- Club Membership and Event Codes ---
1. Register event code
2. View all unique codes
3. Check whether a code exists
4. Remove a code
5. Show total unique codes
6. Exit
Enter your choice (1-6): 4
Enter code to remove: English101
Code not found.
```

ACTIVITY 03 | Library Book Lookup and Borrowing Queue

Problem Statement

A small library must quickly locate books using unique accession codes and process borrowing requests in arrival order. Build one console application that maintains both book records and a queue of pending requests.

Required Java Collections

HashMap<String,String> and Queue<String> implemented by ArrayDeque<String> (both required); use put(), get(), containsKey(), entrySet(), offer(), peek(), poll().

Required Menu

1. Add book (code and title). 2. Find book by code. 3. List books. 4. Add borrowing request (student name and book code). 5. View next request. 6. Process next request. 7. Exit.

Functional Requirements

- Reject duplicate book codes and blank book titles.
- Only queue a request if the requested book code exists.
- Display a request with enough information to identify the student and book.
- Handle an empty request queue without an exception.
- Processing a request removes it from the queue; it does not automatically delete the book record.

Test Scenarios (demonstrate all) add screenshot for each testcases and corresponding output.

1. Add B101 → "Java Basics" and B102 → "Data Structures"; search both.

2. Attempt to add B101 again and verify duplicate rejection.

3. Queue Ana/B101, Ben/B102; process one request and verify Ben is next.

4. Try to request B999 and process an empty queue.

Open-Ended Questions

1. Why is a Map better than searching an ArrayList of titles for a unique book code?

A HashMap uses keys to look up data instantly ($O(1)$ time), whereas an ArrayList would force you to loop through every single element sequentially to find a match.

2. What does FIFO mean, and why does it matter for fair request processing?

FIFO stands for First-In, First-Out. It ensures fairness by making sure the person who requested first gets helped first.

3. How would you modify the design to track available copies and prevent borrowing an unavailable book?

Instead of mapping a string title directly, you could create a small Book object that holds both the title and an integer for available copies, and check/decrement that number whenever someone requests it.

```
PS C:\Collections0070L>
PS C:\Collections0070L>  c:; cd 'c:\Collections0070L'; & 'C:
.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExcep
ceStorage\ebc84a7405279c317ca37d9b2d6b0628\redhat.java\jdt_

--- Library System ---
1. Add book (code and title)
2. Find book by code
3. List books
4. Add borrowing request (student name and book code)
5. View next request
6. Process next request
7. Exit
Enter your choice (1-7): 1
Enter book code: B101
Enter book title: Java Basics
Book added successfully.

--- Library System ---
1. Add book (code and title)
2. Find book by code
3. List books
4. Add borrowing request (student name and book code)
5. View next request
6. Process next request
7. Exit
Enter your choice (1-7): 1
Enter book code: B102
Enter book title: Data Structures
Book added successfully.

--- Library System ---
1. Add book (code and title)
2. Find book by code
3. List books
4. Add borrowing request (student name and book code)
5. View next request
6. Process next request
7. Exit
Enter your choice (1-7): 2
Enter code to search: B101
Found -> Code: B101, Title: Java Basics

--- Library System ---
1. Add book (code and title)
2. Find book by code
3. List books
4. Add borrowing request (student name and book code)
5. View next request
6. Process next request
7. Exit
Enter your choice (1-7): 2
Enter code to search: B102
Found -> Code: B102, Title: Data Structures
```

```
--- Library System ---
1. Add book (code and title)
2. Find book by code
3. List books
4. Add borrowing request (student name and book code)
5. View next request
6. Process next request
7. Exit
Enter your choice (1-7): 1
Enter book code: B101
Duplicate code! Book already exists.
```

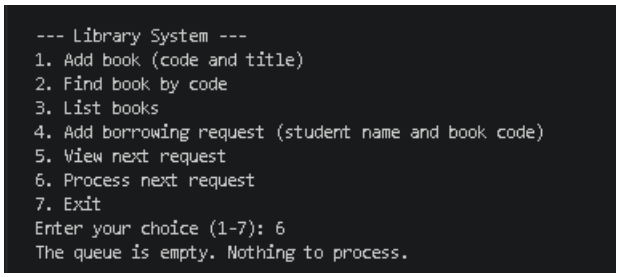
```
--- Library System ---
1. Add book (code and title)
2. Find book by code
3. List books
4. Add borrowing request (student name and book code)
5. View next request
6. Process next request
7. Exit
Enter your choice (1-7): 4
Enter student name: Ana
Enter requested book code: B101
Borrowing request queued.

--- Library System ---
1. Add book (code and title)
2. Find book by code
3. List books
4. Add borrowing request (student name and book code)
5. View next request
6. Process next request
7. Exit
Enter your choice (1-7): 4
Enter student name: Ben
Enter requested book code: B102
Borrowing request queued.
```

```
--- Library System ---
1. Add book (code and title)
2. Find book by code
3. List books
4. Add borrowing request (student name and book code)
5. View next request
6. Process next request
7. Exit
Enter your choice (1-7): 6
Processed request: Student: Ana | Book Code: B101 (Java Basics)

--- Library System ---
1. Add book (code and title)
2. Find book by code
3. List books
4. Add borrowing request (student name and book code)
5. View next request
6. Process next request
7. Exit
Enter your choice (1-7): 5
Next request: Student: Ben | Book Code: B102 (Data Structures)
```

```
--- Library System ---
1. Add book (code and title)
2. Find book by code
3. List books
4. Add borrowing request (student name and book code)
5. View next request
6. Process next request
7. Exit
Enter your choice (1-7): 4
Enter student name: Jian
Enter requested book code: B999
That book code doesn't exist. Request rejected.
```



Submission Checklist and Assessment

☐ Three separately compilable .java files (one public class per file).

☐ Each program includes a numbered menu, Scanner input, repetition until Exit, and input validation.

☐ All specified collection classes and operations are demonstrated.

☐ Screenshots or terminal logs show the listed test scenarios.

☐ All nine open-ended questions are answered in complete sentences.